

The Name of the Title Is Hope

FirstName Surname1stAutor
DepartmentName,
Institution/UniversityName
City, Country
email@email.com

FirstName Surname
DepartmentName,
Institution/UniversityName
City, Country
email@email.com

FirstName Surname
DepartmentName,
Institution/UniversityName
City, Country
email@email.com

FirstName Surname
DepartmentName,
Institution/UniversityName
City, Country
email@email.com

FirstName Surname
DepartmentName,
Institution/UniversityName
City, Country
email@email.com

FirstName Surname
DepartmentName,
Institution/UniversityName
City, Country
email@email.com

FirstName Surname
DepartmentName,
Institution/UniversityName
City, Country
email@email.com

FirstName Surname
DepartmentName,
Institution/UniversityName
City, Country
email@email.com

FirstName Surname
DepartmentName,
Institution/UniversityName
City, Country
email@email.com

Abstract

This ACM-like template, derived from the original acmart document class, is intended to support the preparation of scientific papers for the symposia and workshops that compose CBSOFT. It preserves the visual identity and structural organization of the ACM consolidated format while adapting its elements to comply with the editorial and formatting guidelines adopted by the Brazilian Computer Society. This template can be used for both review and camera-ready versions, allowing authors to prepare their manuscripts with minimal structural changes. The abstract text must be written as a single continuous paragraph, without paragraph breaks.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

1 Introdução

Diante dos recentes avanços de modelos de IA (inteligência artificial), a Engenharia de Software vem se modificando para se adequar tais práticas. As ferramentas baseadas em IA tem auxiliado a apoiar a sociedade em tarefas cotidianas e do trabalho, dentro do ciclo de desenvolvimento, podemos apontar uma ajuda na licitação de requisitos, geração de código, testes e documentação.

De forma mais recente, agentes autônomos de programação tem se demonstrado capaz de implementar funcionalidades completas a partir de uma descrição usando linguagem natural, reduzindo assim o esforço manual necessário dos desenvolvedores de software.

Apesar dos avanços tecnológicos e da chegada de novos modelos poderosos, ainda é possível observar que a qualidade dos artefatos produzidos por agentes depende diretamente da qualidade das instruções fornecidas para eles. Requisitos incompletos, ambíguos e com uma estrutura errada podem gerar resultados inconsistentes durante o processo de desenvolvimento, aumentando o retrabalho e o número de interações necessárias para concluir tal tarefa. Por conta disso, é necessário utilizar técnicas avançadas de Engenharia

de Prompt e refinamento de requisitos, para melhorar a comunicação entre desenvolvedores humanos e agentes baseados em Large Language Models (LLM).

Paralelo a isso, observa-se o surgimento de diversas abordagens diferentes para conseguirmos utilizar tais LLMs, dentre elas a utilização de múltiplos agentes, na qual cada um assume responsabilidade específica dentro do processo de desenvolvimento. Nesse cenário, um agente pode atuar com o refinamento dos requisitos enquanto outro é responsável pela implementação do código, reproduzindo de forma simplificada os papéis tradicionais que vemos nos profissionais do mercado de trabalho.

Este trabalho irá investigar uma abordagem de desenvolvimento orientada por agentes baseados na separação das atividades de refinamento de requisito e implementação de software. Como base para o estudo, foi desenvolvido um software para auxiliar a produção de textos acadêmicos utilizando $\text{\LaTeX}$, cujas funcionalidades e elementos de interface foram concebidos por meio de um processo de desenvolvimento assistido por agentes. A proposta consiste em utilizar um agente especializado em transformar as descrições e requisitos de funcionalidades escritas por humanos em prompts robustos que posteriormente serão utilizados como entrada para um agente autônomo de programação responsável por tal implementação de solução.

Como objetivo do trabalho, busca-se analisar o potencial dessa estratégia para apoiar o desenvolvimento de software assistido por IA. Serão discutidos seus benefícios, limitações e implicações para a Engenharia de Software. Para isso, foi conduzido um estudo de caso no qual os requisitos da aplicação forma inicialmente definidos por um desenvolvedor humano, refinados por um agente especializado e posteriormente implementados por um agente autônomo de programação.

A organização desse artigo está dividido em 4 seções, no qual a Seção 2 apresentaremos os conceitos fundamentais relacionados a

agentes de software. A Seção 3 descreveremos a abordagem proposta e o estudo de caso realizado. Por fim, a Seção 4 apresenta as conclusões e perspectivas para trabalhos futuros.

2 Fundamentação teórica

2.1 LLMs na Engenharia de Software

As Large Language Models (LLMs) são modelos de inteligência artificial (IA) treinados sobre grandes volumes de dados textuais com objetivo de compreender e gerar linguagem natural. Com base na arquitetura Transformer, esses modelos foram capazes de aprender e se especializar por meio de padrões linguísticos e relações semânticas que lhe permitem realizar tarefas como geração de texto, tradução, respostas a perguntas complexas e produção de código. Alguns exemplos utilizados atualmente incluem o GPT, Claude, Gemini e Llama.

No meio da Engenharia de Software esses modelos foram introduzidos inicialmente como assistentes para geração de pequenos trechos de código, passando a apoiar diversas atividades ao longo do ciclo de desenvolvimento. Na Engenharia de Requisitos, por sua vez, auxiliam na elicitación, refinamento e validação dos requisitos.

Com o avanço da tecnologia com ajuda do Machine Learning os LLMs começaram a atuar de forma mais autônoma, diretamente do terminal do computador de um desenvolvedor. Desse modo, assumiram papéis especializados para execução das tarefas, colaborando para execução de tarefas complexas de desenvolvimento de software.

Dentre os benefícios observados na literatura, observasse o aumento da produtividade dos desenvolvedores, redução do tempo necessário para execução de tarefas repetitivas, apoio na documentação, auxílio na geração de testes e identificações de problema e democratização do acesso e conhecimento técnico.

2.2 Engenharia de Prompt

A Engenharia de Prompt (EP) consiste na criação de um texto elaborado propondo instruções estruturada para orientar as LLMs na execução de tarefas específicas. A qualidade e a clareza no prompt está diretamente ligada com a consistência e a relevância das respostas geradas.

Diversas técnicas tem sido propostas para melhorar o desempenho das respostas. Entre elas destacam-se o *Role Prompting*, no qual o modelo recebe um papel específico a ser desempenhado, como analista de requisitos, PO ou arquiteto de software. O *Chain-of-Thought*, no qual incentiva a geração de um raciocínio intermediário para analisar e pensar como será feita a resposta final. O *Structured Prompting*, baseado na definição no formato estruturado de entrada e saída, reduzindo ambiguidade e tornando respostas mais consistentes. Por ultimo, o *Few-Shot Prompting*, que se utiliza de exemplos para demonstrar o comportamento esperado do modelo.

2.3 Sistemas Multiagentes Baseados em LLMs

Os sistemas multiagentes consiste na utilização de múltiplos agentes especializados na realização conjunta de uma tarefa, de forma que atinjam um objetivo comum.

Na Engenharia de Software, a prática desses sistemas tem se tornado cada vez mais utilizada para distribuir responsabilidades entre especialistas de IA. Sua abordagem busca produzir divisão

de trabalho observada em equipes de desenvolvimento humanas, permitindo com que cada agente atue em uma parte da tarefa.

A combinação entre Engenharia de Prompt e sistemas multiagentes possibilita a construção de fluxos de desenvolvimento onde um agente consegue refinar os requisitos, produzindo um artefato que serve como entrada para agentes responsáveis pela implementação do software.

3 Proposta

A abordagem principal da proposta está composta em 3 etapas fazendo uma alusão com que vemos no mercado de trabalho. Utilizamos o auxílio de 2 agentes especializados para atuarem como o Engenheiro de Requisitos da aplicação e o Engenheiro de Software, tendo como papel de PO o usuário que está orquestrando eles.

O primeiro agente, se utilizou o modelo GPT-5.5, onde foi responsável pelo refinamento e detalhamento dos requisitos passados pelo usuário. Seu objetivo foi transformar descrições informais de funcionalidade em especificações detalhadas e estruturadas no formato de prompts para ser utilizada na etapa de desenvolvimento.

O segundo agente foi responsável por pensar na implementação a partir da descrição detalhada dos requisitos. Sua execução foi feita a partir da plataforma visual OpenCode utilizando o modelo Mimo 2.5. Esse agente por sua implementou as funcionalidades solicitadas, realizando criação ou edição no código-fonte da aplicação, produzindo os artefatos necessários para atender aos requisitos definidos.

Desse modo, o fluxo proposto se baseia nas responsabilidades reais do mercado de trabalho, buscando produzir uma divisão de responsabilidades observadas em equipe de desenvolvimento de software. No entanto, faz-se necessário o conhecimento em programação para entender e especificar para a LLM de geração de requisitos a proposta de segurança, bibliotecas utilizadas e meios para alcançar tais objetivos.



Figure 1: Pipeline de desenvolvimento orientado por agentes para refinamento de requisitos e implementação de software

3.1 Agente Refinador de Requisitos

O agente refinador tem como objetivo transformar solicitações informais e com baixo aceite pelas LLMs geradoras de código em estruturas mais detalhadas e específicas, auxiliando o desenvolvimento assistido por IA. Para isso, uma descrição textual elaborada identificando todos os objetivos de uma funcionalidade, objetivos, regras de negócio e fluxos principais e critérios de aceite devem ser passado para o modelo utilizado para refinar.

Todos os textos devolvidos pelo o modelo de refinação devem ser validados e interpretados para entender o entendimento completo da funcionalidade, evitando com que ela seja desenvolvida incompleta ou de forma incorreta. Um ponto de destaque, é a importância de existir um profissional técnico por trás da criação desse texto inicial, para conduzir o texto refinado no modelo que ele construiria a aplicação, comentando as bibliotecas usadas, validações necessárias e outros pontos que podem impactar no desenvolvimento e na segurança do projeto.

No nosso caso empírico, a escolha pela utilização do GPT se deu pela sua alta capacidade criativa e de escrita, se mostrando extremamente habilidoso na escrita dos prompts. Entretanto, entende-se que qualquer agente capaz de escrever prompts com qualidade podem ser usado nessa etapa de desenvolvimento.

3.2 Agente Desenvolvedor

Após o refinamento dos requisitos e especificações de implementação, o prompt gerado deve ser utilizado como entrada para o agente de desenvolvimento utilizando o modo *thinking* para que ocorra o pensamento e criação do fluxo das tarefas refinadas pela outra LLM. A partir desse momento, o agente poderá efetuar questionamentos para o desenvolvedor e esses devem ser analisados e respondidos conforme a necessidade do projeto.

Todos os retornos de código devem ser analisados e verificados por um desenvolvedor com capacidade crítica de entender se o que foi feito possui manutenibilidade, confiabilidade e segurança. As implementações devem seguir o padrão na qual seriam feitas pelo próprio desenvolvedor, com isso ele deve ter total domínio da linguagem de programação utilizada para compreender a estrutura criada.

Um ponto de destaque é a importância de um arquivo com instruções para o agente se tornar mais técnico e específico no projeto, podendo ser um *AGENTS.md* na raiz, especificando toda a arquitetura do projeto para que ele siga os padrões pré definidos. O guia de arquitetura, funcionalidades, cores e skills utilizadas dentro desse arquivo funcionará como um manual de instruções para ele se basear e entender como deve estruturar as funcionalidades. Com tudo, devemos novamente refinar esse manual utilizando um modelo de inteligência artificial generativo para deixar descritivo e claro.

Dentro do projeto do editor em latex, foi proposto a utilização da ferramenta Open-Source do opencode com o mímico 2.5, por conta do seu baixo consumo por tokens e a capacidade de pensamento avançado disponível como opção antes da execução do desenvolvimento. Entretanto é possível se utilizar de outras ferramentas e modelos para utilização dessa técnica, tendo em vista a necessidade de um desenvolvedor experiente e com conhecimento na área para tomar a decisão de qual utilizar no projeto.

3.3 Aplicação e Observações

Para avaliar a aplicabilidade da metodologia proposta, foi desenvolvida uma ferramenta de edição de textos acadêmicos. A ferramenta tem como objetivo auxiliar o processo de escrita, organização e revisão de documentos, integrando recursos tradicionais de edição com funcionalidades baseadas em inteligência artificial.

As principais funcionalidades implementadas foram:

- Criação, importação e clonagem de projetos a partir de repositórios Git;
- Listagem e gerenciamento dos projetos criados pelo usuário;
- Interface gráfica para edição de arquivos .tex, contendo recursos de leitura e manipulação do código-fonte;
- Criação, edição e organização de subpastas e arquivos dentro do projeto;
- Importação e gerenciamento de imagens utilizadas na composição dos documentos;
- Reorganização de arquivos e diretórios por meio de operações de arrastar e soltar (*drag and drop*);
- Visualização do PDF compilado diretamente na aplicação;
- Navegação por seções do documento, permitindo acesso rápido a capítulos e subseções por meio de âncoras;
- Download do documento PDF gerado;
- Inicialização de repositórios Git nos projetos e execução de comandos básicos de versionamento (em fase de aprimoramento para correção de inconsistências);
- Apresentação de mensagens de erro detalhadas durante o processo de compilação, facilitando a identificação e resolução de problemas;
- Integração com modelos de inteligência artificial para auxílio na revisão textual, incluindo correção ortográfica, análise de concordância verbal, melhoria de sentenças e identificação de repetição excessiva de termos. Foram utilizados modelos locais baseados em Llama e modelos disponibilizados pela plataforma Groq, permitindo que o usuário selecione o provedor desejado por meio de variáveis de ambiente;
- Desenvolvimento da documentação técnica do projeto utilizando a ferramenta MkDocs;
- Implementação de pipeline de integração e entrega contínua (CI/CD) para geração e publicação automática da documentação via GitHub Pages após alterações no repositório principal.

Como trabalhos futuros e itens presentes no backlog de desenvolvimento, destacam-se:

- Implementação de uma análise completa do projeto utilizando inteligência artificial para identificar e sugerir correções ortográficas e gramaticais em todos os arquivos do documento, sem depender do arquivo atualmente aberto pelo usuário;
- Utilização de inteligência artificial para avaliar a aderência do conteúdo desenvolvido aos requisitos definidos pela banca avaliadora, identificando possíveis lacunas ou melhorias necessárias;
- Desenvolvimento de um recurso de assistência ao usuário para esclarecer dúvidas relacionadas à utilização de recursos $\LaTeX$, incluindo auxílio na aplicação de comandos de formatação textual, como negrito, itálico e outros elementos de estruturação do documento.

4 Citations and Bibliographies

The use of BibTeX for the preparation and formatting of one's references is strongly recommended. Authors' names should be complete. Use full first names ("Donald E. Knuth") instead of initials ("D. E. Knuth"), and the identifying features of a reference must be included: title, year, volume, number, pages, article DOI, etc.

The bibliography is included in your source document with these two commands, placed just before the `\end{document}` command:

```
\bibliographystyle{ACM-Reference-Format}
\bibliography{bibfile}
```

`bibfile` is the BibTeX filename without the `.bib` suffix.

Citations and references are numbered by default. Some examples:

- A paginated journal article [2]
- An enumerated journal article [11]
- A reference to an entire issue [10]
- A monograph (whole book) [24]
- A monograph/whole book in a series [18]
- A divisible book such as an anthology or compilation [13]
- A divisible book (series not included since no volume number is given) [14]
- A chapter in a divisible book [36]
- A chapter in a divisible book in a series [12]
- A multi-volume work as a book [23]
- Paginated proceedings articles [3, 16]
- A proceedings article with all possible elements [35]
- An enumerated proceedings article [15]
- An informally published work [17]
- Preprints [6, 8]
- A doctoral dissertation [9]
- A master's thesis [4]
- Online documents / World Wide Web resources [1, 28, 37]
- Video games (Case 1) [27]
- Video games (Case 2) [25, 26]
- A patent (Case 3) [34]
- Work accepted for publication [31]
- 'YYYYb' test for prolific author [32, 33]
- Citations containing duplicate DOI and URLs (e.g., some SIAM articles) [22]
- Multi-volume works as books [19, 20]
- A presentation [30]
- An article under review [7]
- Citations with DOIs [21, 22]
- Online citations [37–39]
- Artifacts [5, 29]

5 Recomme that demonstratependices

If your work needs an appendix, add it before the `\end{document}` command at the conclusion of your source document.

Start the appendix with the `appendix` command:

```
\appendix
```

In the appendix, sections are lettered rather than numbered. This document has two appendices that demonstrate the section and subsection identification method.

6 Conclusion and Future Work

<text included by the authors>

Artifact Availability

This section must be included immediately after the Conclusion and before the Acknowledgments section. It must not be numbered and must be titled exactly as **Artifact Availability**.

If the paper provides research artifacts (such as datasets, source code, scripts, models, or experimental packages), the authors must describe what is available, where it can be accessed, and under which license or usage conditions.

If no artifacts are available, this section must still be included, explicitly stating that no artifacts are provided.

For papers written in Portuguese, this section must be titled **Disponibilidade de Artefatos**.

Acknowledgements

<text included by the authors>

References

- [1] Rafal Ablamowicz and Bertfried Fauser. 2007. *CLIFFORD: a Maple 11 Package for Clifford Algebra Computations, version 11*. Retrieved February 28, 2008 from <http://math.tntech.edu/rafal/cliff11/index.html>
- [2] Patricia S. Abril and Robert Plant. 2007. The patent holder's dilemma: Buy, sell, or troll? *Commun. ACM* 50, 1 (Jan. 2007), 36–44. doi:10.1145/1188913.1188915
- [3] Sten Andler. 1979. Predicate Path expressions. In *Proceedings of the 6th. ACM SIGACT-SIGPLAN symposium on Principles of Programming Languages (POPL '79)*. ACM Press, New York, NY, 226–236. doi:10.1145/567752.567774
- [4] David A. Anisi. 2003. *Optimal Motion Control of a Ground Vehicle*. Master's thesis. Royal Institute of Technology (KTH), Stockholm, Sweden.
- [5] Sam Anzaroot and Andrew McCallum. 2013. *UMass Citation Field Extraction Dataset*. Retrieved May 27, 2019 from <http://www.iesl.cs.umass.edu/data/data-umasscitationfield>
- [6] Sam Anzaroot, Alexandre Passos, David Belanger, and Andrew McCallum. 2014. *Learning Soft Linear Constraints with Application to Citation Field Extraction*. arXiv:1403.1349 doi:10.48550/arXiv.1403.1349
- [7] R. Baggett, M. Simecek, C. Chambellan, K. Tsui, and M. Fraune. 2025. Fluidity in the Phased Framework of Technology Acceptance: Case Study to Gain a Holistic Understanding of (Older Adult) Participant Advancement Through Acceptance Phases with Mobile Telepresence Robots. *Robotics Aut. Systems*. Manuscript submitted for review.
- [8] Lutz Bornmann, K. Brad Wray, and Robin Haunschild. 2019. *Citation concept analysis (CCA)—A new form of citation analysis revealing the usefulness of concepts for other researchers illustrated by two exemplary case studies including classic books by Thomas S. Kuhn and Karl R. Popper*. arXiv:1905.12410 [cs.DL]
- [9] Kenneth L. Clarkson. 1985. *Algorithms for Closest-Point Problems (Computational Geometry)*. Ph. D. Dissertation. Stanford University, Palo Alto, CA. UMI Order Number: AAT 8506171.
- [10] Jacques Cohen (Ed.). 1996. Special issue: Digital Libraries. *Commun. ACM* 39, 11 (Nov. 1996).
- [11] Sarah Cohen, Werner Nutt, and Yehoshua Sagie. 2007. Deciding equivalences among conjunctive aggregate queries. *J. ACM* 54, 2, Article 5 (April 2007), 50 pages. doi:10.1145/1219092.1219093
- [12] Bruce P. Douglass, David Harel, and Mark B. Trakhtenbrot. 1998. Statecarts in use: structured analysis and object-orientation. In *Lectures on Embedded Systems*, Grzegorz Rozenberg and Frits W. Vaandrager (Eds.). Lecture Notes in Computer Science, Vol. 1494. Springer-Verlag, London, 368–394. doi:10.1007/3-540-65193-4_29
- [13] Ian Editor (Ed.). 2007. *The title of book one* (1st. ed.). The name of the series one, Vol. 9. University of Chicago Press, Chicago, Chapter The title of the chapter, 127–238. doi:10.1007/3-540-09237-4
- [14] Ian Editor (Ed.). 2008. *The title of book two* (2nd. ed.). University of Chicago Press, Chicago, Chapter 100, 25–137. doi:10.1007/3-540-09237-4
- [15] Matthew Van Gundy, Davide Balzarotti, and Giovanni Vigna. 2007. Catch me, if you can: Evading network signatures with web-based polymorphic worms. In *Proceedings of the first USENIX workshop on Offensive Technologies (WOOT '07)*. USENIX Association, Berkeley, CA, Article 7, 9 pages.

- [16] Torben Hagerup, Kurt Mehlhorn, and J. Ian Munro. 1993. Maintaining Discrete Probability Distributions Optimally. In *Proceedings of the 20th International Colloquium on Automata, Languages and Programming (Lecture Notes in Computer Science, Vol. 700)*. Springer-Verlag, Berlin, 253–264.
- [17] David Harel. 1978. *LOGICS of Programs: AXIOMATICS and DESCRIPTIVE POWER*. MIT Research Lab Technical Report TR-200. Massachusetts Institute of Technology, Cambridge, MA.
- [18] David Harel. 1979. *First-Order Dynamic Logic*. Lecture Notes in Computer Science, Vol. 68. Springer-Verlag, New York, NY. doi:10.1007/3-540-09237-4
- [19] Lars Hörmander. 1985. *The analysis of linear partial differential operators. III*. Grundlehren der Mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences], Vol. 275. Springer-Verlag, Berlin, Germany. viii+525 pages. Pseudodifferential operators.
- [20] Lars Hörmander. 1985. *The analysis of linear partial differential operators. IV*. Grundlehren der Mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences], Vol. 275. Springer-Verlag, Berlin, Germany. vii+352 pages. Fourier integral operators.
- [21] IEEE 2004. IEEE TCSC Executive Committee. In *Proceedings of the IEEE International Conference on Web Services (ICWS '04)*. IEEE Computer Society, Washington, DC, USA, 21–22. doi:10.1109/ICWS.2004.64
- [22] Markus Kirschmer and John Voight. 2010. Algorithmic Enumeration of Ideal Classes for Quaternion Orders. *SIAM J. Comput.* 39, 5 (Jan. 2010), 1714–1747. doi:10.1137/080734467
- [23] Donald E. Knuth. 1997. *The Art of Computer Programming, Vol. 1: Fundamental Algorithms (3rd. ed.)*. Addison Wesley Longman Publishing Co., Inc., Boston.
- [24] David Kosior. 2001. *Understanding Policy-Based Networking* (2nd. ed.). Wiley, New York, NY.
- [25] Newton Lee. 2005. Interview with Bill Kinder: January 13, 2005. Video. *Comput. Entertain.* 3, 1, Article 4 (Jan.-March 2005). doi:10.1145/1057270.1057278
- [26] Dave Novak. 2003. Solder man. Video. In *ACM SIGGRAPH 2003 Video Review on Animation theater Program: Part I - Vol. 145 (July 27–27, 2003)*. ACM Press, New York, NY, 4. doi:10.945/woot07-S422 <http://video.google.com/videoplay?docid=6528042696351994555>
- [27] Barack Obama. 2008. A more perfect union. Video. Retrieved March 21, 2008 from <http://video.google.com/videoplay?docid=6528042696351994555>
- [28] Poker-Edge.Com. 2006. Stats and Analysis. Retrieved June 7, 2006 from <http://www.poker-edge.com/stats.php>
- [29] R Core Team. 2019. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org/>
- [30] Brian J. Reiser. 2014. Designing coherent storylines aligned with NGSS for the K-12 classroom. Presentation at National Science Education Leadership Association Meeting, Boston, MA, USA. <https://www.academia.edu/6884962/>
- [31] Bernard Rous. 2008. The Enabling of Digital Libraries. *Digital Libraries* 12, 3, Article 5 (July 2008). To appear.
- [32] Mehdi Saeedi, Morteza Saheb Zamani, and Mehdi Sedighi. 2010. A library-based synthesis methodology for reversible logic. *Microelectron. J.* 41, 4 (April 2010), 185–194.
- [33] Mehdi Saeedi, Morteza Saheb Zamani, Mehdi Sedighi, and Zahra Sasanian. 2010. Synthesis of Reversible Circuit Using Cycle-Based Approach. *J. Emerg. Technol. Comput. Syst.* 6, 4 (Dec. 2010), 12 pages.
- [34] Joseph Scientist. 2009. The fountain of youth. Patent No. 12345, Filed July 1st., 2008, Issued Aug. 9th., 2009.
- [35] Stan W. Smith. 2010. An experiment in bibliographic mark-up: Parsing metadata for XML export. In *Proceedings of the 3rd. annual workshop on Librarians and Computers (LAC '10, Vol. 3)*. Reginald N. Smythe and Alexander Noble (Eds.). Paparazzi Press, Milan Italy, 422–431.
- [36] Asad Z. Spector. 1990. Achieving application requirements. In *Distributed Systems* (2nd. ed.), Sape Mullender (Ed.). ACM Press, New York, NY, 19–33. doi:10.1145/90417.90738
- [37] Harry Thornburg. 2001. *Introduction to Bayesian Statistics*. Retrieved March 2, 2005 from <http://ccrma.stanford.edu/~jos/bayes/bayes.html>, archived at [<https://web.archive.org/web/20240505055615/https://ccrma.stanford.edu/~jos/bayes/bayes.html>]
- [38] TUG 2017. *Institutional members of the T_EX Users Group*. Retrieved May 27, 2017 from <http://wwtug.org/instmem.html>
- [39] Boris Veytsman. 2017. *acmart—Class for typesetting publications of ACM*. Retrieved May 27, 2017 from <http://www.ctan.org/pkg/acmart>

A Appendix Example 1

A.1 Part One

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi malesuada, quam in pulvinar varius, metus nunc fermentum urna, id sollicitudin purus odio sit amet enim. Aliquam ullamcorper eu

ipsum vel mollis. Curabitur quis dictum nisl. Phasellus vel semper risus, et lacinia dolor. Integer ultricies commodo sem nec semper.

A.2 Part Two

Etiam commodo feugiat nisl pulvinar pellentesque. Etiam auctor sodales ligula, non varius nibh pulvinar semper. Suspendisse nec lectus non ipsum convallis congue hendrerit vitae sapien. Donec at laoreet eros. Vivamus non purus placerat, scelerisque diam eu, cursus ante. Etiam aliquam tortor auctor efficitur mattis.

B Appendix Example 2

Nam id fermentum dui. Suspendisse sagittis tortor a nulla mollis, in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

Nam interdum magna at lectus dignissim, ac dignissim lorem rhoncus. Maecenas eu arcu ac neque placerat aliquam. Nunc pulvinar massa et mattis lacinia.